Prince Sultan University

College of Computer and Information Sciences / Department of Computer Science

Term 261

CS102 Computer Programming II

RiyadhPoly Project

Phase 1 — Design and Board

Deadline: Oct 24, 2026 on the LMS, followed by a 5-minute check-in in the next lab session (2 marks). Read the *Overview* first. By the end of this phase, your program loads `board.txt`, checks it, and prints the board. Players and money come in Phase 2.

What to build

- ☐ A **class diagram** of your whole design (on paper or with a tool). It may change later; bring it to the check-in.
- ☐ `ColorGroup`: `public static final String` constants `BROWN`, `YELLOW`, `GREEN`, `BLUE`, `RAILROAD`.
- ☐ `Square` (abstract) with an index, a name and `abstract void land(Player p, Game g)`.
- ☐ Its 7 subclasses: `GoSquare`, `ChanceSquare`, `ParkSquare`, `JailSquare`, `GoToJailSquare`, `PropertySquare`, `StationSquare`. In this phase, `land` may just print a message such as `landed on Olaya`.
- ☐ `Ownable` (interface), implemented by `PropertySquare` and `StationSquare`: price, rent, group, owner.
- ☐ `InvalidBoardException`: a checked exception (it extends `Exception`).
- ☐ `Board`: reads `board.txt` into an `ArrayList<Square>`, builds the color-group `Map`, and prints the board.
- ☐ Empty `Player` and `Game` classes, so the code compiles. You fill them in Phase 2.

Store every fixed number as a `static final` constant, starting with `BOARD_SIZE = 20`.

The board file

The program reads the board from `board.txt` at startup. Nothing about squares (names, prices, rents, positions) may be hard-coded. **Your instructor will give your group its own board file**, and your program must work with any valid file, including the one used in the viva.

Each line is one square: `index,type,name,group,price,rent`. Lines starting with `#` are comments. Squares without a price leave the last three fields empty. The default board is:

```
# index,type,name,group,price,rent
0,GO,Go,,,
1,PROPERTY,Al-Shemaisy,BROWN,100,40
2,PROPERTY,Al-Bathaa,BROWN,100,48
3,CHANCE,Chance,,,
4,PROPERTY,Al-Oud,BROWN,120,56
5,GO_TO_JAIL,Go To Jail,,,
6,PROPERTY,Al-Tahlia,YELLOW,200,100
7,STATION,Central Station,RAILROAD,300,100
8,PROPERTY,Olaya,YELLOW,220,112
9,PROPERTY,King Fahd,YELLOW,250,128
10,PARK,Park,,,
11,PROPERTY,KAFD,GREEN,400,200
12,PROPERTY,Al-Nakheel,GREEN,450,220
13,CHANCE,Chance,,,
14,PROPERTY,Al-Aqiq,GREEN,500,240
15,JAIL,Jail,,,
16,PROPERTY,Kingdom Centre,BLUE,650,300
17,STATION,Grand Terminal,RAILROAD,300,100
18,PROPERTY,Rafal,BLUE,700,320
19,PROPERTY,Diriyah,BLUE,800,360
```

Validation. If the file is missing, or a line is invalid, throw an `InvalidBoardException` with the line number and the reason, then print a clear message and stop. A line is invalid if:

- it has the wrong number of fields
- its type or group is unknown
- its price or rent is not a number (catch the `NumberFormatException` and throw an `InvalidBoardException` instead)

Example messages:

- `Error in board.txt, line 8: unknown group "WHITE".`
- `Error in board.txt, line 13: price "four hundred" is not a number.`

Tip: `line.split(",", -1)` keeps the empty fields at the end of a line such as `0,GO,Go,,,`. Without `-1`, Java drops them.

Code hints

These are starting points, not complete code. Fields, constructors, getters and `toString` are up to you.

```
public abstract class Square {
    private final int index;
    private final String name;
    // constructor, getters, toString ...

    public abstract void land(Player p, Game g);   // each subclass decides what
happens
}

public interface Ownable {
    int getPrice();
    int getBaseRent();
    String getGroup();
    Player getOwner();
    void setOwner(Player p);   // null means the bank owns it
}
```

`Board` keeps the squares in order and the color groups in a `Map`. Fill both while reading the file. When a line is a `PROPERTY` or `STATION`, add the square to its group's list:

```
public class Board {
    private ArrayList<Square> squares = new ArrayList<>();
    private Map<String, ArrayList<Ownable>> groups = new HashMap<>();

    public Board(String fileName) throws InvalidBoardException { ... }

    // While loading a property or station:
    //     groups.putIfAbsent(group, new ArrayList<>());
    //     groups.get(group).add(square);

    public int countOwned(Player p, String group) {
        int count = 0;
        for (Ownable o : groups.get(group)) {
            if (o.getOwner() == p) count++;
        }
        return count;
    }
}
```

You will use `countOwned` in Phase 2. A player owns a whole group when `countOwned(p, g) == groups.get(g).size()`.

Creating the right square. Read the type field and create the matching subclass, for example with a `switch` on `"GO"`, `"PROPERTY"`, `"STATION"` and so on. An unknown type is an `InvalidBoardException`.

Test it, then the check-in

Before the deadline, check that:

- ☐ The default `board.txt` loads and your program prints all 20 squares, for example `8: Olaya (YELLOW) price $220, rent $112`.
- ☐ Your group's own board file also loads.
- ☐ Three broken files each give a clear error message, not a crash: one with a missing field, one with an unknown group, and one with a non-numeric price.

- ☐ `countOwned` returns 0 for every group (nobody owns anything yet).

Check-in (5 minutes, 2 marks). Show your class diagram and the printed board, then load a broken board file. The instructor then asks **each member** one short question about the code, for example "Why is `Square` abstract?" or "Why does `Board` use a `Map`?".

Earns	Marks
Diagram matches the code; the board loads, prints and rejects a broken file	1.5 (group)
Your answer to your question	0.5 (individual)

Next: *Phase 2 — Players and Money*.